

Lesson 8: Introduction to Web Scraping

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Explore a web page's DOM
I Do	10–15 min	HTML basics, Selenium, CSS selectors, ethics
We Do	15–20 min	Scrape Wikipedia links together
You Do	5–10 min	Find elements on a page
Wrap-Up	5 min	Assignment preview & capstone intro

Warm-Up (5 min)

Inspect a Web Page

1. Open any website in Chrome
2. Right-click on a heading → **Inspect**
3. Look at the **Elements** tab in Developer Tools

Find these things:

- The `<title>` tag
- An `<a>` tag (link) — what's its `href` attribute?
- A `<p>` tag — what text does it contain?

Share what you found in the chat.

I Do: Why Web Scraping?

There's a lot of data on the internet — but it's trapped in web pages meant for humans.

Web scraping extracts that data into a structured format you can analyze.

Real-world examples:

- Comparing prices across airline websites
- Collecting research article metadata
- Gathering public meeting schedules
- Building a dataset that doesn't exist as a CSV

Scraping turns **unstructured web pages** into **structured data** (CSV, JSON, database).

I Do: HTML and the DOM

HTML is the structure of every web page. The **DOM** is its tree:

```
<html>
  <head>
    <title>My Page</title>
  </head>
  <body>
    <h1>Welcome</h1>
    <p>Hello world</p>
    <a href="https://example.com">A link</a>
  </body>
</html>
```

I Do: HTML and the DOM (Cont.)

Each element has:

- A **tag** (`p`, `h1`, `a`, `div`)
- **Attributes** (`href`, `class`, `id`)
- **Content** (text or child elements)

I Do: Selenium — The Scraping Tool

Selenium drives a real browser — it handles JavaScript, dynamic content, and Ajax:

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By

driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install())
)
driver.get("https://en.wikipedia.org/wiki/Web_scraping")
print(driver.title)
driver.quit()
```

I Do: Selenium — The Scraping Tool (Cont.)

Why Selenium? Modern web pages load data with JavaScript. A simple HTTP request won't get the full page — Selenium runs the JavaScript just like a real browser.

I Do: Finding Elements with CSS Selectors

CSS selectors let you find elements in the page:

```
# By tag name
paragraphs = driver.find_elements(By.CSS_SELECTOR, "p")

# By class
items = driver.find_elements(By.CSS_SELECTOR, ".result-item")

# By ID
header = driver.find_element(By.CSS_SELECTOR, "#main-header")

# By attribute
images = driver.find_elements(By.CSS_SELECTOR, "img[src]")

# Combined
links = driver.find_elements(By.CSS_SELECTOR, "div.content a")
```

I Do: Finding Elements with CSS Selectors (Cont.)

Selector	Matches
<code>p</code>	All <code><p></code> elements
<code>.classname</code>	Elements with that class
<code>#id</code>	Element with that ID
<code>div a</code>	<code><a></code> inside <code><div></code>

I Do: Getting Data from Elements

Once you find an element, extract its data:

```
# Get the visible text
element.text

# Get an attribute value
element.get_attribute("href")
element.get_attribute("src")
element.get_attribute("class")

# Search within an element (narrow your scope)
parent = driver.find_element(By.CSS_SELECTOR, ".results")
children = parent.find_elements(By.CSS_SELECTOR, "li")
```

Pattern:

1. Find a container element
2. Search *within* it for specific children

I Do: Ethical Web Scraping

Before scraping ANY site, check `robots.txt` :

```
https://example.com/robots.txt
```

`robots.txt` tells you:

- Which pages you're allowed to scrape
- Which pages are off-limits
- How frequently you should make requests

Other ethical guidelines:

- **Add delays** between requests (`time.sleep(2)`)
- **Don't overload** the server with rapid requests
- **Respect** the site's terms of service

I Do: Handling Errors and Headless Mode

Always wrap `driver.get()` in try/except:

```
try:
    driver.get("https://example.com")
    # ... extract data ...
except Exception as e:
    print(f"Error: {type(e).__name__}: {e}")
finally:
    driver.quit()
```

I Do: Handling Errors and Headless Mode (Cont.)

Run without a visible browser window (headless):

```
options = webdriver.ChromeOptions()
options.add_argument("--headless")

driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=options
)
```

Use headless mode in scripts. Use the visible browser while debugging.

I Do: XPath for Tricky Navigation

When CSS selectors aren't enough, use XPath:

```
# Go up to the parent element
parent = element.find_element(By.XPATH, "..")

# Find the next sibling div
next_div = parent.find_element(
    By.XPATH, "following-sibling::div"
)
```

When to use XPath:

- You need to go **up** the tree (parent)
- You need a **sibling** element (no direct CSS path)
- The element has no useful class or ID

CSS selectors cover 90% of cases. XPath handles the tricky 10%.

We Do: Scrape Wikipedia Links

Let's extract all links from the "See also" section together.

Step 1: Set up and load the page:

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By

driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install())
)
driver.get("https://en.wikipedia.org/wiki/Web_scraping")
```

Step 2: Find our starting point:

```
see_also = driver.find_element(
    By.CSS_SELECTOR, '[id="See_also"]'
)
```

We Do: Navigate and Extract

Step 3: Use XPath to find the links container:

```
parent = see_also.find_element(By.XPATH, "..")
links_div = parent.find_element(
    By.XPATH, "following-sibling::div"
)
```

We Do: Navigate and Extract (Cont.)

Step 4: Extract all links:

```
links = []
for a in links_div.find_elements(By.CSS_SELECTOR, "a"):
    name = a.text.strip()
    url = a.get_attribute("href")
    if name and url:
        links.append({"name": name, "url": url})
        print(f"{name}: {url}")

driver.quit()
```

Discussion: Why do we check `if name and url`?

Some link elements might be empty or have no href — always validate.

We Do: Save the Scraped Data

Step 5: Save to CSV and JSON:

```
import csv
import json

# Save to CSV
with open("scraped_links.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "URL"])
    for link in links:
        writer.writerow([link["name"], link["url"]])

# Save to JSON
with open("scraped_links.json", "w") as f:
    json.dump({"links": links}, f, indent=4)
```

The scraping pattern: **load** → **find** → **extract** → **save**.

We Do: The Scraping Workflow

- | | |
|-----------------------------|--|
| 1. Check robots.txt | → Am I allowed? |
| 2. Open dev tools | → Where is the data? |
| 3. Initialize Selenium | → Set up the driver |
| 4. Load the page | → <code>driver.get(url)</code> |
| 5. Find container elements | → CSS selectors |
| 6. Navigate if needed | → XPath for tricky cases |
| 7. Extract text/attributes | → <code>.text</code> , <code>.get_attribute()</code> |
| 8. Store in a list of dicts | → Structured data |
| 9. Save to CSV/JSON | → Persistent storage |
| 10. Close the driver | → <code>driver.quit()</code> |

Follow this workflow for every scraping task.

You Do: Find Elements Challenge (5 min)

Open the browser dev tools on any website and find:

1. The page `<title>` — what does it say?
2. An element with a **class** attribute — what's the class?
3. A link (`<a>`) — what's the `href` ?
4. An image (`<img>`) — what's the `src` ?

Write the CSS selector you'd use to find each one.

Assignment Preview

Task	What You'll Do
1	Check <code>robots.txt</code> for the Durham Library site
2	Use dev tools to find title, author, format elements
3	Write <code>get_books.py</code> — scrape search results
4	Save results to CSV and JSON
5	Analyze Wikipedia's <code>robots.txt</code>
6	Scrape OWASP Top 10 vulnerabilities (uses XPath)
7	Start your web scraping capstone project!

Assignment Tips

Task 2 (DOM exploration): Use Chrome dev tools. Click the arrow next to a `div` to expand it. Keep drilling down until you find the `li` element for a single search result.

Task 3 (get_books.py): Build incrementally:

1. First, just get the page and print the title
2. Then find all `li` elements and print the count
3. Then extract just the title from each `li`
4. Then add author and format

Task 6 (OWASP): You'll need XPath to navigate from the heading to the links.

Capstone Project #2 Kickoff

Starting this week, you're building a **Web Scraping Dashboard**:

Week	What You Build
8 (this week)	Scrape data + clean + save to CSV
9	Store in SQLite database
10	Advanced SQL queries
11	Streamlit interactive dashboard

Recommended site: Weather Around The World (timeanddate.com)

Create a new GitHub repo for this project — it's separate from `python_homework`.

Web Scraping Fragility

Warning: Scraping programs are **fragile**.

- Websites change their HTML structure without notice
- Your selectors may stop working overnight
- Different pages on the same site may have different layouts

Best practices:

- Use broad selectors when possible
- Add error handling for missing elements
- Log errors so you know when something breaks
- Don't rely on deeply nested selectors

Wrap-Up

Today we covered:

- HTML structure and the DOM tree
- Selenium + WebDriver for browser automation
- CSS selectors and XPath for finding elements
- Extracting text and attributes from elements
- Ethical scraping: `robots.txt`, delays, error handling
- Saving scraped data to CSV and JSON

Quick check: Before scraping any website, what's the first thing you should do?

Getting Help This Week

- **Browser dev tools** are your best friend — practice with Inspect
- Build your scraper **one step at a time** — don't try to do everything at once
- Selenium docs: selenium.dev/documentation/webdriver
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for help with selectors or XPath

Next week: Introduction to Databases and SQL — storing your data properly!

Time to Scrape!

The web is full of data — now you know how to get it.

See you next session!